

The Convergence of Mechanics and Intelligence: A Comprehensive Analysis of State-of-the-Art Financial Time Series Forecasting

1. Introduction: The Structural Transformation of Quantitative Forecasting

The discipline of financial time series forecasting is currently navigating a profound paradigm shift, characterized by the dissolution of the historical boundary between statistical econometrics and deep learning. For decades, the field was bifurcated. On one side stood the interpretable, parsimonious world of stochastic calculus and econometrics—dominated by Autoregressive Integrated Moving Average (ARIMA) models, Generalized Autoregressive Conditional Heteroskedasticity (GARCH) frameworks, and the Black-Scholes paradigm. These models offered theoretical elegance and statistical rigor but often struggled to capture the high-dimensional, non-linear, and chaotic dependencies inherent in modern electronic markets. On the other side lay the emerging domain of machine learning—initially Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks—which promised universal approximation capabilities but suffered from opacity, overfitting, and a lack of alignment with fundamental financial theory.

In the 2024-2025 horizon, this dichotomy has largely collapsed. We are witnessing the emergence of a "Symbiotic Intelligence" in quantitative finance. The current State of the Art (SOTA) is defined by architectures that fuse the structural inductive biases of econometrics (e.g., mean reversion, volatility clustering, leverage effects) with the representational power of modern AI (e.g., Transformers, State Space Models, Diffusion Processes).

This report provides an exhaustive technical analysis of this new landscape. We explore how **Foundation Models** like FinCast are attempting to solve the "zero-shot" forecasting problem by learning universal temporal patterns across billions of data points.¹ We dissect the **State Space Renaissance**, where Mamba and S6 architectures are challenging Transformers by offering linear computational complexity, crucial for processing tick-level data.² We analyze the role of **Generative AI**, particularly Diffusion models, in moving beyond point forecasts to full density estimation and synthetic data generation.³ Furthermore, we investigate **Hybrid Econometrics**, where deep learning components like Kolmogorov-Arnold Networks (KANs) are embedded directly into GARCH-MIDAS frameworks to capture non-linear macro-finance linkages.⁴ Finally, we examine the frontier of **Autonomous Agents**, where Large Language Models (LLMs) equipped with layered memory systems are beginning to reason about market

regimes rather than merely predicting the next tick.⁵

This analysis is not merely a catalog of algorithms; it is an investigation into how these mathematical innovations are reshaping the epistemological foundations of market prediction, risk management, and portfolio construction.

2. Foundation Models in Finance: The Quest for Universal Forecasters

The concept of the "Foundation Model"—a large-scale neural network pre-trained on vast, diverse datasets to perform a wide range of downstream tasks—has revolutionized Natural Language Processing (NLP) and Computer Vision (CV). In the domain of time series, this revolution has arrived in the form of models capable of "Zero-Shot" forecasting: predicting the future of a financial instrument (e.g., a newly listed crypto token or an illiquid emerging market stock) without ever having seen it during training.

2.1 FinCast: Anatomy of a Domain-Specific Foundation Model

While general-purpose time series foundation models like TimeGPT, TimesFM, and Chronos have demonstrated impressive capabilities on generic datasets (weather, traffic, electricity), financial data possesses unique "stylized facts" that generic models often fail to capture. These include extreme non-stationarity (distribution shift), low signal-to-noise ratios, and complex multi-resolution dependencies (interaction between intraday volatility and quarterly macro trends).

FinCast¹ represents the SOTA response to these challenges. Unlike its predecessors, FinCast is not just a Transformer applied to numbers; it is a specialized architecture trained on over 20 billion financial time points across diverse asset classes (stocks, futures, forex, crypto).

2.1.1 Architectural Innovations for Financial Nuance

The architecture of FinCast is a decoder-only Transformer, scaled to approximately 1 billion parameters, but its true innovation lies in its specialized components designed to handle the specific idiosyncrasies of market data:

1. Input Tokenization and Learnable Frequency Embeddings:

Financial data is inherently multi-resolution. A sequence of 1-minute close prices has a fundamentally different statistical character (mean-reverting noise) compared to a sequence of monthly GDP prints (smooth trends). Standard positional encodings in Transformers (Sine/Cosine) fail to capture this semantic distinction. FinCast addresses this by normalizing input series and mapping them to latent representations via a residual Multi-Layer Perceptron (MLP). Crucially, it injects learnable frequency embeddings. These embeddings allow the model to contextually understand the resolution of the data it is processing. This enables the model to apply different attention patterns depending

- on whether it is forecasting high-frequency volatility or low-frequency macro trends.¹
2. **Sparse Mixture-of-Experts (MoE) Backbone:**
 The diversity of financial markets is staggering. The dynamics driving a meme stock are distinct from those driving a Treasury bond. A dense model, where every parameter is used for every input, struggles to avoid "interference" between these conflicting patterns. FinCast employs a token-level sparse Mixture-of-Experts (MoE) mechanism.
 - *Mechanism:* The model consists of multiple parallel "expert" subnetworks (Feed-Forward Networks). For each incoming token (patch of time series data), a routing mechanism dynamically selects only the top-k most relevant experts to process that token.
 - *Implication:* This allows the model to scale its parameter count massively (increasing capacity) while keeping inference costs low (linear scalability). It enables the model to develop specialized "sub-brains"—one expert might specialize in mean-reverting regimes, while another specializes in momentum breakouts.¹
 3. **Point-Quantile Loss Function:**
 In finance, being "roughly right" about the distribution is infinitely more valuable than being "precisely wrong" about the mean. Standard Mean Squared Error (MSE) loss functions encourage models to predict the conditional mean, often resulting in smooth, conservative forecasts that fail to capture tail risks. FinCast optimizes a dual objective:
 - *Point Loss:* To ensure accuracy of the central tendency.
 - *Quantile Loss:* To estimate the full predictive distribution (e.g., the 10th and 90th percentiles).
 This alignment with probabilistic forecasting makes the model's output directly usable for risk metrics like Value-at-Risk (VaR).¹

Table 1: Comparative Performance of FinCast vs. Baselines (Zero-Shot)

Metric	FinCast (SOTA)	TimesFM (Google)	Chronos (Amazon)	PatchTST (Supervised)
MSE Reduction	-20%	Baseline	+2.5%	+5.1%
MAE Reduction	-23%	Baseline	+1.8%	+4.2%
Inference Speed	High (MoE)	Medium	Low (Quantized)	High
Probabilistic	Yes (Quantile)	No	Yes (Token)	No

Data synthesized from.¹ Note: Negative reduction implies improvement over baseline.

2.2 The Debate: Continuous vs. Categorical Modeling

A significant philosophical divergence exists in the design of time series foundation models.

The "Language" Approach (Chronos):

Models like Chronos 7 treat time series forecasting as a classification problem. They quantize continuous values into a fixed vocabulary of discrete tokens (bins). This allows them to use off-the-shelf LLM architectures (like T5 or LLaMA) without modification.

- *Pros:* It allows the model to learn arbitrary distributions (multimodal) without assuming a parametric form (like Gaussian). It leverages the massive pre-training of language models.
- *Cons:* Quantization induces a loss of precision. In financial markets, the difference between a price of 100.01 and 100.05 can be the entire profit margin of a trade. Discretization can obscure these micro-structure details.⁷

The "Signal" Approach (TimesFM, FinCast):

Models like TimesFM 9 and FinCast treat time series as continuous signals. They output continuous values directly.

- *Pros:* Preserves infinite precision. Better suited for regression tasks and calculating precise derivatives (Greeks).
- *Cons:* Requires specialized architectures (Patching) to handle the inputs, as standard LLM tokenizers are designed for text.

2.3 Transfer Learning and Regime Adaptation

Even the most powerful foundation model faces the challenge of **distribution shift**. Financial markets are non-stationary; the statistical properties of 2024 are not the same as 2008. SOTA approaches now utilize **Adapter-Based Transfer Learning**.¹⁰

Instead of fine-tuning the entire billion-parameter model (which is computationally expensive and risks "catastrophic forgetting"), researchers freeze the pre-trained weights and insert small, trainable "adapter" layers.

- *Mechanism:* A lightweight adapter module is inserted between the Transformer layers. When the market regime shifts (e.g., from low volatility to high volatility), only the adapter is updated.
- *Empirical Results:* Research on **Tiny Time Mixers (TTM)** demonstrates that adapter-based fine-tuning achieves 25-50% better performance on limited financial datasets compared to training from scratch, effectively mitigating the risk of overfitting to noise.¹⁰

3. The State Space Renaissance: Mamba and Linear Complexity

While Transformers have dominated the last five years, they suffer from a critical flaw: **Quadratic Complexity**. The attention mechanism requires calculating the interaction between every pair of time steps, leading to $O(L^2)$ computational cost. For a financial time series of tick data (where a day might contain 20,000 ticks), this is prohibitively expensive.

This bottleneck has triggered a renaissance in **State Space Models (SSMs)**, spearheaded by the **Mamba** architecture and the **Structured State Space (S4)** paradigm. These models offer the modeling power of Transformers with the linear $O(L)$ scaling of Recurrent Neural Networks (RNNs).¹¹

3.1 MambaStock: The Selective Scan Mechanism (S6)

MambaStock² applies the Mamba architecture specifically to stock price prediction. The core breakdown of why this is SOTA lies in the **Selective Scan Mechanism (S6)**.

Traditional State Space Models (like Kalman Filters or S4) are time-invariant. The matrices governing the state transition (A, B, C) are constant. This means the model processes a quiet afternoon of trading with the same computational "dynamics" as a market crash. This is inefficient.

The S6 Innovation:

Mamba makes the state transition parameters functions of the input.

$$h_t = A h_{t-1} + B(x_t) x_t$$

$$y_t = C(x_t) h_t$$

(Simplified notation)

By making B and C dependent on the current input x_t , the model gains the ability to **Selectivity**:

1. **Gating**: It can choose to let relevant information (a sudden price shock) enter the hidden state h_t aggressively.
2. **Forgetting**: It can choose to reset the state or ignore noise during periods of irrelevant consolidation.

This "Input-Dependent Dynamics" allows MambaStock to achieve the content-aware selectivity of a Transformer's attention mechanism but compiles it down to a recurrent scan operation that runs in linear time.² Empirical studies show MambaStock outperforming

Transformers on high-frequency datasets while requiring significantly less GPU memory.¹²

3.2 Probabilistic Mamba: Mamba-ProbTSF

A deterministic forecast is of limited utility in finance. Traders need to know the variance (risk).

Mamba-ProbTSF¹³ extends the architecture to probabilistic forecasting.

- **Dual-Network Topology:** The model runs two parallel Mamba backbones.
 - *Mean Network:* Predicts the expected future value μ_t .
 - *Variance Network:* Predicts the uncertainty σ_t^2 (aleatoric uncertainty).
- **Performance:** In benchmark tests on volatile series (like electricity load, a proxy for volatility), the true trajectory falls within the model's predicted confidence intervals 95% of the time. This validates Mamba's ability to model not just the level of the series but its changing volatility structure.¹³

3.3 Graph-Mamba: Modeling Inter-Asset Dependencies

Financial assets are highly correlated. The price of an airline stock depends on the price of oil.

Graph-Mamba¹² combines the temporal efficiency of Mamba with the spatial reasoning of Graph Neural Networks (GNNs).

- **Structure:** Each stock is a node in a graph. The edges represent correlations or supply chain links.
- **Process:** A Mamba layer processes the time series of each node individually to extract temporal features. Then, a Graph Convolutional layer aggregates information from neighbor nodes (e.g., propagating an oil price shock to airline stocks). This hybrid architecture captures the spatiotemporal diffusion of information across the market.

4. Generative AI in Finance: Diffusion Models and Synthetic Reality

The introduction of **Denoising Diffusion Probabilistic Models (DDPMs)**—the technology behind image generators like DALL-E—to finance marks a shift from "Discriminative" modeling (predicting y given x) to "Generative" modeling (learning the joint probability $P(x,y)$).³

4.1 Diffolio: Probabilistic Portfolio Construction

Diffolio¹⁴ is a groundbreaking application of diffusion to multivariate forecasting. In portfolio optimization, the "Killer App" is estimating the covariance matrix of assets, especially during tail events when correlations tend to spike (correlation breakdown).

Mechanism:

Diffolio treats the future price path of a portfolio not as a single line, but as a distribution of possible paths. It learns to generate these paths by reversing a noise process.

1. **Forward Process:** Gradually adds Gaussian noise to historical return sequences until

they are pure noise.

2. **Reverse Process (Generative):** A neural network learns to denoise the sequence, conditioned on past market data and covariates.

Hierarchical Attention with Correlation Guidance:

Diffolio introduces a novel regularizer. During the generation process, it enforces a structural constraint based on a stable estimate of the asset correlation matrix. This guides the diffusion process to generate scenarios that are not just individually realistic, but jointly coherent.

- **Result:** Portfolios optimized using Diffolio's generated scenarios achieve a **Sharpe Ratio of 0.72**, significantly outperforming baselines like TimeGAN (0.05-0.07) and standard Diffusion-TS. It excels particularly in the "Growth Optimal Portfolio" (GOP) metric, which is highly sensitive to tail risks.¹⁴

4.2 Visualizing Market Microstructure: LOB-as-Image

A fascinating convergence of Computer Vision and Finance is the representation of **Limit Order Books (LOB)** as images. The LOB contains the hierarchy of buy and sell orders at different price levels. By visualizing this as a 2D heatmap (Price $\times$ Time), researchers can apply Diffusion Models with **Inpainting** techniques.¹⁵

- **Task:** Given the "image" of the order book for the last 10 minutes, "paint" the next 1 minute.
- **Why it works:** Diffusion models are exceptional at capturing spatial textures and structures. In LOB images, "texture" corresponds to liquidity density and "structure" corresponds to order flow imbalances. This approach captures complex microstructure patterns (like liquidity layering or spoofing) that tabular models miss.¹⁵

4.3 Synthetic Data for Regime Stress Testing

One of the greatest challenges in finance is the scarcity of crisis data. We have limited examples of "2008-style" crashes. Diffusion models are now SOTA for **Synthetic Data Generation**. By training a DDPM on historical crises, quants can generate infinite variations of "counterfactual" crises.¹⁶

- **Stylized Facts Preservation:** Unlike GANs (Generative Adversarial Networks), which often suffer from mode collapse (generating the same sample repeatedly), Diffusion models produce diverse samples that accurately replicate the fat tails and volatility clustering of real markets. This allows for robust stress-testing of trading algorithms against scenarios that have never happened but *could* happen.

5. Hybrid Econometrics: The Neural-Symbolic Synthesis

While Deep Learning excels at pattern recognition, it lacks the structural interpretability of

Econometrics. The most robust systems in 2025 are **Hybrids** that embed econometric logic directly into neural architectures.

5.1 KAN-GARCH-MIDAS: Non-Linear Volatility Decomposition

The GARCH-MIDAS model is a staple of macro-finance. It decomposes volatility into a short-term component (g_t , driven by daily returns) and a long-term component (τ_t , driven by low-frequency macro variables like GDP via MIDAS polynomials).

$$\sigma_t^2 = g_t \times \tau_t$$

The KAN Innovation:

The KAN-GM model ⁴ replaces the linear or polynomial MIDAS filter with a Kolmogorov-Arnold Network (KAN).

- *KAN vs. MLP*: Standard Multi-Layer Perceptrons (MLPs) place activation functions on the *nodes*. KANs place learnable activation functions on the *edges*. This is based on the Kolmogorov-Arnold representation theorem, which states that any multivariate continuous function can be represented as a superposition of continuous functions of one variable.
- *Application*: The KAN learns the complex, non-linear mapping from macro variables (Inflation, Interest Rates) to the long-term volatility trend τ_t .
- *Performance*: KAN-GM outperforms traditional GARCH-MIDAS and MLP-based hybrids in Mean Absolute Error (MAE) and cross-market adaptability (Stocks vs. Forex). It effectively captures the non-linear "leverage effect" where bad macro news spikes volatility more than good news reduces it.⁴

5.2 GARCH-GRU: Inductive Bias in Recurrent Cells

Another powerful hybrid is the **GARCH-GRU**.¹⁸ Instead of treating the RNN as a black box, this architecture modifies the internal calculation of the Gated Recurrent Unit (GRU).

- *Mechanism*: The GRU cell is augmented with an explicit GARCH(1,1) update equation. The hidden state of the RNN is forced to carry information about the conditional variance σ_t^2 .
- *Inductive Bias*: By hard-coding the GARCH logic into the network, the model is "forced" to respect the phenomenon of volatility clustering. It cannot "forget" that high volatility today implies high volatility tomorrow.
- *Result*: This model achieves lower MSE than pure GRU or pure GARCH models, specifically in small-data regimes where pure deep learning models tend to overfit.¹⁸

6. Microstructure and Self-Excitation: Hawkes Processes

In the realm of High-Frequency Trading (HFT), the assumption of discrete time steps (e.g.,

1-second bars) breaks down. Markets move in "event time"—trades occur irregularly in microseconds. **Hawkes Processes** are the SOTA for modeling this asynchronous event flow.¹⁹

6.1 Neural Hawkes Processes

A Hawkes process is a "self-exciting" point process. The intensity (probability of an event) $\lambda(t)$ is boosted by past events:

$$\lambda(t) = \mu + \sum_{t_i < t} \phi(t - t_i)$$

where ϕ is the decay kernel (usually exponential).

Modern Advances:

- **Neural Parametrization:** Standard Hawkes processes assume a fixed decay kernel (exponential). **Neural Hawkes Processes** use an LSTM or Transformer to learn the kernel function ϕ from data. This captures non-monotonic decay patterns (e.g., a burst of activity 50ms after a trade due to latency arbitrage algorithms).²⁰
- **Multivariate Modeling:** These models track the mutual excitation between different order types (Limit Buy, Market Sell). A market sell order might "excite" a limit buy order cancellation (liquidity flight).
- **Rough Volatility Link:** Recent theoretical work establishes a bridge between micro-scale Hawkes processes and macro-scale **Rough Volatility**. As the aggregation of many self-exciting order flows converges, it generates a volatility process driven by Fractional Brownian Motion with $H < 0.5$ (Rough Volatility).²¹ This allows traders to estimate macro-roughness parameters directly from tick data.

6.2 Engineering Ecosystem: Sparklen and Tick

The implementation of these models requires specialized engineering. Python loops are too slow for calculating the recursive intensity of Hawkes processes.

- **Sparklen:** A new Python toolkit²² that wraps a high-performance C++ core. It focuses on high-dimensional Hawkes processes (hundreds of assets) using sparse regularization (Lasso) to identify the "Granger Causal" links between order flows.
- **HawkesPyLib:** Focuses on simulation and inference, using JIT compilation (Numba) to accelerate the Ogata thinning algorithm used for simulation.²³

7. End-to-End Decision Making: Differentiable Optimization

The "Predict-then-Optimize" pipeline—where one model predicts returns and a separate optimizer allocates the portfolio—is fundamentally flawed. The prediction model optimizes for MSE (accuracy), but the investor cares about Sharpe Ratio (risk-adjusted return). Small errors

in prediction can lead to disastrous portfolios if they occur in correlated assets.

End-to-End (Decision-Focused) Learning integrates the optimization problem *into* the neural network's training loop.

7.1 BPQP: Differentiable Convex Optimization Layers

The challenge in End-to-End learning is backpropagation. If the output of a network is the solution to a Quadratic Program (QP) (finding optimal weights w^*), how do we compute the gradient of the loss $\mathcal{L}$ with respect to the network weights θ ?

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\partial \mathcal{L}}{\partial w^*} \times \frac{\partial w^*}{\partial \theta}$$

The term $\frac{\partial w^*}{\partial \theta}$ requires differentiating through the argmin operator.

BPQP (Backward Pass as Quadratic Programming):

Standard methods (like OptNet) use implicit differentiation of the KKT conditions, which involves solving a large linear system (computing the inverse Hessian). This is slow ($O(N^3)$).

BPQP reformulates the backward pass itself as a Quadratic Program.

- *Advantage:* It avoids explicit Hessian inversion. It enables the use of efficient first-order solvers for the gradient computation.
- *Scale:* BPQP allows for the training of Deep Portfolios on high-dimensional universes (e.g., S&P 500) an order of magnitude faster than OptNet. It enables the network to learn "implicit constraints" (e.g., which assets tend to become illiquid) that are not explicitly coded in the optimizer.²⁵

7.2 Decision by Supervised Learning (DSL)

An alternative to full differentiability is **Decision by Supervised Learning (DSL)**.²⁶

- *Concept:* Instead of solving the optimization problem during inference, the model learns to *predict the optimal weights*.
- *Training:* A "Oracle" computes the true optimal weights for historical data (ex-post). A neural network (Deep Ensemble) is trained to predict these oracle weights using a convex surrogate loss (like Cross-Entropy).
- *Stability:* This method avoids the instability of training against non-convex financial objectives (like Sharpe Ratio) directly via Reinforcement Learning. It acts as a "distillation" of the optimization logic into a neural network, allowing for millisecond inference times.

8. Autonomous Financial Agents: The Reasoning

Frontier

The integration of **Large Language Models (LLMs)** into trading goes beyond sentiment analysis. The SOTA involves **Autonomous Agents** that reason, plan, and execute.

8.1 Layered Memory and Reflective Cognition

Financial context decays at different rates. A price tick is relevant for seconds; an earnings report for weeks; a rate hike for months. Standard LLM context windows are "flat"—they treat all tokens as equally accessible.

FinMem and TradingGPT⁵ introduce **Layered Memory Systems**:

1. **Short-Term Memory**: A circular buffer storing recent ticks and news headlines. High decay rate.
2. **Working Memory**: Stores active orders and current strategy state.
3. **Long-Term Memory**: A vector database (RAG) storing historical analogues (e.g., "How did the market react to the 2008 Lehman collapse?").

Reflective Loop:

These agents employ a "Profile-Based" architecture. Different instances of the LLM adopt different personas (e.g., "Risk Manager," "Momentum Trader," "Macro Analyst"). They engage in a structured debate (Chain-of-Thought) to reach a consensus trade. This "Reflective" step significantly reduces the hallucination rate compared to a single LLM deciding alone.⁵

8.2 StockBench: The Reality Check

Despite the hype, the **StockBench** benchmark²⁸ provides a sobering reality check.

- *Finding*: Most generic LLMs (GPT-4, Claude 3) fail to beat a simple Buy-and-Hold strategy in terms of risk-adjusted returns.
- *Why*: LLMs struggle with precise numerical reasoning (arithmetic) and often "hallucinate" trends that don't exist.
- *Solution*: The SOTA agents are **Tool-Augmented**. The LLM does not predict the price; it writes Python code to call a reliable forecaster (like Mamba or ARIMA) and then interprets the result. The LLM acts as the "Orchestrator," not the "Calculator."

9. Deep Hedging and Neural SDEs

For derivatives (Options, Futures), the goal is not prediction but **Hedging** (replicating the payoff to eliminate risk).

9.1 Rough Volatility and Deep Calibration

Standard hedging (Black-Scholes) assumes volatility is a smooth process. Real volatility is Rough (Hölder regular with $H \approx 0.1$). This means volatility paths look like "dust" rather than smooth curves.

Rough Bergomi (rBergomi) models capture this but are non-Markovian, making them computationally intractable to calibrate using standard methods (simulation takes too long).

Deep Calibration:

Researchers train Neural Networks to approximate the map from Model Parameters (Θ) to Option Prices (P).

$$\mathcal{N}(\Theta) \approx \text{Price}(\Theta)$$

Once trained, the network can be inverted (or used in an optimizer) to find parameters that fit market prices in milliseconds. This enables the use of sophisticated Rough Volatility models in real-time trading desks.²⁹

9.2 Neural SDEs for Hedging

Deep Hedging³¹ replaces the mathematical derivation of the hedge ratio (Delta) with a neural network.

- *Training:* The network observes a simulated market (which can be a Rough Volatility model or a GAN). It outputs a trading action. The loss function is the **CVaR (Conditional Value at Risk)** of the final P&L.
- *Result:* The network learns to hedge "frictions" like transaction costs and liquidity gaps that mathematical formulas must ignore to remain solvable. It discovers strategies (like "overshooting" the hedge to compensate for lag) that human traders learn by intuition.³²

10. Benchmarking Crisis: FinTSB and Standardization

A critical issue in the field has been the "Standardization Deficit." Papers often cherry-pick datasets or metrics. **FinTSB**³³ is the SOTA response.

FinTSB Framework:

1. **Regime Classification:** It categorizes datasets not just by asset, but by *behavior*: "Uptrend," "Downtrend," "Oscillation," and "Extreme" (Black Swan).
2. **Holistic Metrics:** It mandates reporting on three levels:
 - *Signal Quality:* IC (Information Coefficient).
 - *Strategy Quality:* Sharpe Ratio, Max Drawdown.
 - *Regression Quality:* MSE/MAE.
3. **Cost Simulation:** It explicitly models transaction costs. A model with 99% accuracy that trades 1000 times a day might lose money due to fees. FinTSB exposes this.³⁵

11. Conclusion: The Era of Symbiotic Finance

The trajectory of financial forecasting is clear. The era of "pure" econometrics is over, but the era of "pure" black-box AI is also ending. The future belongs to **Symbiotic Architectures** that

integrate:

1. **Universal Pattern Recognition** via Foundation Models (FinCast).
2. **Efficient State Management** via Mamba/S6.
3. **Probabilistic Rigor** via Diffusion and GARCH hybrids.
4. **Structural Constraints** via Differentiable Optimization (BPQP).

For the practitioner, the implication is that the "Edge" no longer lies in a single proprietary model, but in the engineering capability to orchestrate this complex ecosystem of Neural, Symbolic, and Probabilistic components into a coherent decision engine.

Citations

1

Works cited

1. FinCast: A Foundation Model for Financial Time-Series Forecasting - arXiv, accessed January 8, 2026, <https://arxiv.org/html/2508.19609v1>
2. MambaStock: Selective state space model for stock prediction - arXiv, accessed January 8, 2026, <https://arxiv.org/html/2402.18959v1>
3. Diffusion Models for Time Series Forecasting: A Survey - arXiv, accessed January 8, 2026, <https://arxiv.org/html/2507.14507v2>
4. Full article: Combining deep learning with econometric models: volatility forecasting using the KAN-GARCH-MIDAS framework - Taylor & Francis Online, accessed January 8, 2026, <https://www.tandfonline.com/doi/full/10.1080/15140326.2025.2555479>
5. LLM Trading Agent Overview - Emergent Mind, accessed January 8, 2026, <https://www.emergentmind.com/topics/llmtradingagent>
6. FinCast: A Foundation Model for Financial Time-Series Forecasting - alphaXiv, accessed January 8, 2026, <https://www.alphaxiv.org/overview/2508.19609v1>
7. Benchmarking Foundation Models for Time-Series Forecasting: Zero-Shot, Few-Shot, and Full-Shot Evaluations - MDPI, accessed January 8, 2026, <https://www.mdpi.com/2813-0324/11/1/32>
8. puffinsoft/benchstreet: The benchmark for financial time series forecasting. - GitHub, accessed January 8, 2026, <https://github.com/puffinsoft/benchstreet>
9. The arrival of foundation models in time series forecasting - PricePedia, accessed January 8, 2026, <https://www.pricepedia.it/en/magazine/article/2025/11/07/the-arrival-of-foundation-models-in-time-series-forecasting/>
10. Time Series Foundation Models for Multivariate Financial Time Series Forecasting - arXiv, accessed January 8, 2026, <https://arxiv.org/html/2507.07296v1>
11. [2402.18959] MambaStock: Selective state space model for stock prediction - arXiv, accessed January 8, 2026, <https://arxiv.org/abs/2402.18959>

12. MambaStock: Selective state space model for stock prediction - IDEAS/RePEc, accessed January 8, 2026, <https://ideas.repec.org/p/arx/papers/2402.18959.html>
13. Mamba time series forecasting with uncertainty propagation - arXiv, accessed January 8, 2026, <https://arxiv.org/html/2503.10873v1>
14. Diffolio: A Diffusion Model for Multivariate Probabilistic ... - arXiv, accessed January 8, 2026, <https://arxiv.org/pdf/2511.07014>
15. Painting the market: generative diffusion models for financial limit order book simulation and forecasting - arXiv, accessed January 8, 2026, <https://arxiv.org/html/2509.05107v1>
16. Full article: Generation of synthetic financial time series by diffusion models, accessed January 8, 2026, <https://www.tandfonline.com/doi/full/10.1080/14697688.2025.2528697>
17. [2410.18897] Generation of synthetic financial time series by diffusion models - arXiv, accessed January 8, 2026, <https://arxiv.org/abs/2410.18897>
18. Integrated GARCH-GRU in Financial Volatility Forecasting - arXiv, accessed January 8, 2026, <https://arxiv.org/html/2504.09380v1>
19. Hawkes Models and Their Applications - Annual Reviews, accessed January 8, 2026, <https://www.annualreviews.org/content/journals/10.1146/annurev-statistics-112723-034304>
20. Hawkes Processes Modeling, Inference, and Control: An Overview - ResearchGate, accessed January 8, 2026, https://www.researchgate.net/publication/370620430_Hawkes_Processes_Modeling_Inference_and_Control_An_Overview
21. Discussion Paper Series 2024-E-6 Abstract A Survey of Rough Volatility, accessed January 8, 2026, <https://www.imes.boj.or.jp/research/abstracts/english/24-E-06.html>
22. Sparklen: a statistical learning toolkit for high-dimensional Hawkes processes in Python - arXiv, accessed January 8, 2026, <https://arxiv.org/pdf/2502.18979?>
23. HawkesPyLib documentation, accessed January 8, 2026, <https://simbold.github.io/HawkesPyLib/>
24. BPQP: A Differentiable Convex Optimization Framework for Efficient End-to-End Learning, accessed January 8, 2026, [https://openreview.net/forum?id=VKKY3Uv7vi&referrer=%5Bthe%20profile%20of%20Jiang%20Bian%5D\(%2Fprofile%3Fid%3D~Jiang_Bian1\)](https://openreview.net/forum?id=VKKY3Uv7vi&referrer=%5Bthe%20profile%20of%20Jiang%20Bian%5D(%2Fprofile%3Fid%3D~Jiang_Bian1))
25. BPQP: A Differentiable Convex Optimization Framework for Efficient End-to-End Learning - NIPS papers, accessed January 8, 2026, https://proceedings.neurips.cc/paper_files/paper/2024/file/8db12f7214d3a1a0c450ba751163e0fd-Paper-Conference.pdf
26. Decision by Supervised Learning with Deep Ensembles: A Practical Framework for Robust Portfolio Optimization - arXiv, accessed January 8, 2026, <https://arxiv.org/html/2503.13544v3>
27. Comparing LLM-Based Trading Bots: AI Agents, Techniques, and Results in Automated Trading | FlowHunt, accessed January 8, 2026, <https://www.flowhunt.io/blog/llm-trading-bots-comparison/>

28. StockBench: Can Llm Agents Trade Stocks Profitably In Real-world Markets? - arXiv, accessed January 8, 2026, <https://arxiv.org/html/2510.02209v1>
29. Rough Bergomi: Fractional Volatility & Calibration - Emergent Mind, accessed January 8, 2026, <https://www.emergentmind.com/topics/rough-bergomi-model>
30. Deep calibration of the rough Bergomi model - Mathematical Institute - University of Oxford, accessed January 8, 2026, <https://www.maths.ox.ac.uk/node/29915>
31. pfnet-research/pfhedge: PyTorch-based framework for Deep Hedging - GitHub, accessed January 8, 2026, <https://github.com/pfnet-research/pfhedge>
32. Deep Hedging under Rough Volatility - Research Collection, accessed January 8, 2026, <https://www.research-collection.ethz.ch/server/api/core/bitstreams/fe79064c-a432-4b24-a698-43dfe047961d/content>
33. FinTSB: A Comprehensive and Practical Benchmark for Financial Time Series Forecasting, accessed January 8, 2026, <https://arxiv.org/html/2502.18834v1>
34. TongjiFinLab/FinTSB: FinTSB: A Comprehensive and ... - GitHub, accessed January 8, 2026, <https://github.com/TongjiFinLab/FinTSBenchmark>
35. FinTSB: A Comprehensive and Practical Benchmark for Financial Time Series Forecasting, accessed January 8, 2026, https://www.researchgate.net/publication/389392050_FinTSB_A_Comprehensive_and_Practical_Benchmark_for_Financial_Time_Series_Forecasting
36. Bayesian Dynamic Factor Models for High-Dimensional Matrix-Valued Time Series, accessed January 8, 2026, <https://www.oru.se/contentassets/84372fcaa4784b22b5bd7d15e645a560/zhang--updated.pdf>